

MC3113 Mechatronic Systems Design

Lab 03: V&V Harness for a Sealed Thermal Controller

Student Guide | 24 March 2026 | 08:30 – 12:30

1. Quick Reference

Item	Detail
Date	27 March 2026
Start	13:30
Soft deadline	18:00 (full marks)
Hard deadline	23:00 (–1 mark per 30 min after 18:00)
Marks	10 (LO4: Software Interfaces)
Tools	MATLAB only (no Python, no Simulink)
Submit	Push to the 'lab03' branch of your team repo

Lab 03 carries its own 10 marks, independent of CA3. Your lab mark is not affected by the status of your CA3 submission.

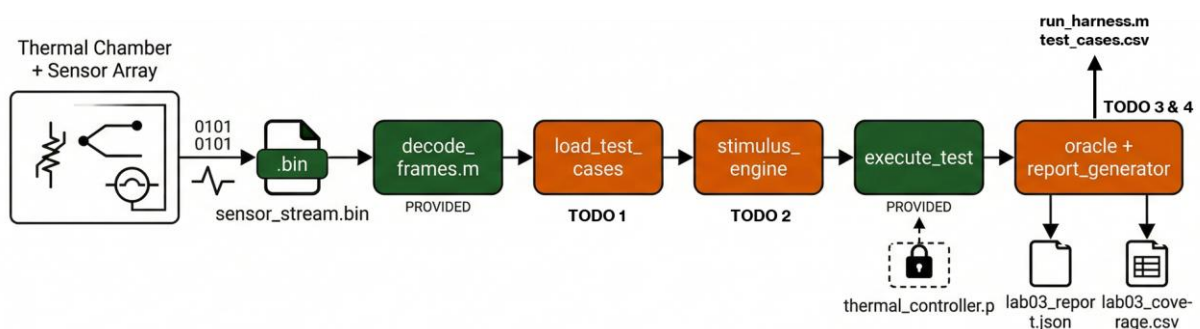
2. The Story: From Sensor to Verdict

Every mechatronic system produces data. In this lab, a thermal chamber, a sealed enclosure that heats and cools to a target temperature, has been running three test scenarios. Its internal sensors have been logging data to a binary file the whole time. That file, *sensor_stream.bin*, is sitting in your lab folder right now.

The chamber is controlled by software: *thermal_controller.p*. It is sealed; you cannot open it, read its source, or modify it. Your job today is not to *build* the controller. Your job is to **test it** to find out whether it behaves correctly and is safe.

To do that, you will build a **V&V (Verification & Validation) harness**: a software pipeline that reads the raw sensor stream, generates the correct test inputs, runs the controller, measures its outputs against defined requirements, and writes a structured report. One of those requirements is a safety limit. By the end of the lab, your harness will tell you whether the sealed controller violates it, and the answer may surprise you.

Here is the complete pipeline you are building. Dark green nodes are provided to you; do not modify them. You build the amber nodes. The pipeline flows strictly in sequence; each stage produces a clean output that the next stage consumes.



Each stage corresponds to one function file in your lab03/ folder. The diagram is your map for the entire session; refer to it whenever you are unsure where your current task fits.

3. Learning Outcomes

By the end of this lab, you will be able to:

- **LLO1** — Operationalize a software requirement into a runnable test case
- **LLO2** — Design a stimulus appropriate to a given metric (step vs. ramp)
- **LLO3** — Implement a test oracle that produces a quantitative verdict
- **LLO4** — Generate a machine-readable test report from structured results
- **LLO5** — Identify an intentional safety defect from oracle output alone

You are **not** implementing the controller, debugging the plant model, or writing Simulink blocks. You are the test engineer.

Important: Scaffolding and Placeholder Code

The function files provided in your lab03/ folder: `load_test_cases.m`, `stimulus_engine.m`, `oracle.m`, and `report_generator.m`, contain **stub code and placeholder return values** (for example, `measured = 0;` or `measured = Inf;`). These placeholders exist solely to prevent MATLAB syntax errors before you begin. **They are not correct implementations.**

Your task for each file is to replace every block marked `% >>> YOUR CODE HERE <<<` with working MATLAB code. The function signatures, variable names, and `% PROVIDED` blocks must not be changed.

Where the guide does not specify an exact implementation detail, you may make reasonable engineering assumptions. **Any such assumption must be documented as a comment** at the point in the code where it is applied, using the format:

`% ASSUMPTION: [brief statement of what was assumed and why]`

For example:

```
% ASSUMPTION: If neither T_10 nor T_90 crossing is found, rise time is
%             returned as Inf, indicating the step response did not complete
%             within the run duration.
```

You do not need to list assumptions anywhere else. The marker will read them in context.

MATLAB Help is your primary reference. For any function used in your implementation (e.g. `find`, `mean`, `interp1`, `jsonencode`), run `doc <function_name>` in the MATLAB Command Window for the official documentation.

4. Before You Start (13:30–13:50)

Step 0A: Verify MATLAB

version *% must report R2021b or later*

If MATLAB is older than R2021b, jsonencode with 'PrettyPrint' will not work. Inform the TA immediately.

Step 0B: Pull lab files

Open a **terminal** (not MATLAB) and run: `git pull origin main`

Then, in MATLAB, set Current Folder to the `.\lab03\` subfolder of your local repo clone.

You must be inside your team's linefollower repo clone for this to work. The lab03/ folder will appear in your clone only after git pull. If it does not appear, confirm you are in the correct repo directory before pulling. All files must be in one folder; do not move any file out of lab03.

Step 0C: Confirm all 13 files are present

```
% Run in MATLAB with Current Folder set to lab03/
lab_files = {
    'sensor_stream.bin',    'decode_frames.m',    'execute_test.m', ...
    'thermal_controller.p', 'verify_task1.m',    'test_cases.csv', ...
    'lab03_scaffold.m',    'load_test_cases.m',  'stimulus_engine.m', ...
    'oracle.m',            'report_generator.m', ...
    'cut_interface_spec.pdf', 'Lab03_Student_Guide.pdf'};

for f = lab_files
    if isfile(f{1}), s = 'OK'; else, s = 'MISSING'; end
    fprintf('%-35s %s\n', f{1}, s);
end
```

Every line must print OK. If anything is MISSING, run `git pull origin main` again and check your Current Folder path.

Step 0D: Run the decode verification

`>> verify_task1`

All 12 checks must print PASS. The key ones:

```
CHECK 3  valid frame count : 3741    PASS
CHECK 6  Run 1 row count   : 863 ±5   PASS
CHECK 7  Run 2 row count   : 1151 ±5  PASS
CHECK 8  Run 3 row count   : 1727 ±5  PASS
```

CHECK	What it tests	If it FAILs
1	crc16.m computes the correct checksum for a known byte sequence	crc16.m is corrupted or missing. Run git pull origin main and confirm crc16.m prints OK in the Step 0C file check. If still failing, ask the TA for a replacement crc16.m.
2	sensor_stream.bin is the correct size (39030 bytes)	Wrong binary file. Do not proceed. Ask the TA for the correct sensor_stream.bin.
3	decode_frames returns exactly 3741 rows	decode_frames.m has a logic error, or CHECK 2 also failed (wrong binary). If CHECK 2 passed, ask the TA; decode_frames.m is a provided file and should not fail.
4	The decoded table has all 4 required columns (run_id, t, t_amb_degC, setpoint_degC)	decode_frames.m has a structural error. Ask the TA; do not attempt to fix a provided file.
5	The decoded table contains exactly 3 distinct run_id values: [1, 2, 3]	The binary contains fewer or more runs than expected. This almost always means CHECK 2 also failed. Ask the TA for the correct sensor_stream.bin.
6–8	Row counts per run are within ± 5 of expected (863, 1151, 1727)	See Fallback Gate below.
9	Run 1 starts at setpoint 60 °C and ambient 20 °C	Data values decoded incorrectly. If CHECK 3 passed, ask the TA — decode_frames.m is provided.
10	Run 2 ends at setpoint ≈ 80 °C (ramp end value)	Same as CHECK 9 — provided file issue. Inform the TA.
11	Run 3 setpoint is sustained at ≈ 95 °C	Same as CHECK 9 — provided file issue. Inform the TA.
12	decoded.csv was successfully written to disk	MATLAB does not have write permission in the current folder, or the disk is full. Try: cd(userpath) then re-run verify_task1. If the problem persists, inform the TA.

General rule: CHECKs 1, 4, 5, 9, 10, and 11 all test provided files (crc16.m, decode_frames.m, sensor_stream.bin) that you did not write and cannot fix. If any of these fail and you cannot resolve it in under two minutes with git pull, inform the TA. Do not spend lab time debugging a provided file.

Step 0E: Read the Controller Under Test (CUT) interface

Read *cut_interface_spec.pdf*. It specifies exactly how *thermal_controller.p* accepts inputs and returns outputs. The key facts:

Field	Direction	Type	Meaning
inputs.setpoint	→ CUT	double	Target temperature [°C]
inputs.ambient_temp	→ CUT	double	Ambient temperature [°C]
inputs.dt	→ CUT	double	Time step [s]
out.chamber_temp	← CUT	double	Simulated chamber temp [°C]
out.heater_cmd	← CUT	double	Heater command [0–1]
out.cooler_cmd	← CUT	double	Cooler command [0–1]
Out.fault_flag	← CUT	int	0 = nominal, 1 = warning, 2 = fault

Note on *fault_flag*: *execute_test.m* does not append *fault_flag* to the log table, so it is not available as a column in *oracle.m*. None of the four required metrics (Tasks 1–4) use *fault_flag*. For Task 5, you may add a test case using any of the four valid metrics; *fault_flag* cannot be used directly as an oracle metric in this lab.

You never call *thermal_controller* directly. *execute_test.m* runs the loop for you; it takes a prepared log table and appends *chamber_temp*, *heater_cmd*, and *cooler_cmd* columns to it.

The Three Test Runs

The binary file *sensor_stream.bin* encodes three distinct scenarios. Each scenario becomes a *run_id* in your decoded table:

run_id	Stimulus	Setpoint	Duration	What it tests
1	Step	60 °C	~86 s	Rise time and steady-state tracking
2	Ramp	80 °C	~115 s	Ramp-following accuracy
3	Sustained fault	95 °C	~173 s	Safety limit compliance

Run 3 is the critical one. The controller is commanded to hold at 95 °C, above the safety threshold of 85 °C, for nearly three minutes. Your oracle will measure what fraction of that time the chamber actually exceeded 85 °C. Keep this in mind as you build.

5. Task 1: Case Loader (13:50–14:20)

What does it do and why

The test harness you are building needs to know *what* to test. `test_cases.csv` defines the test suite: each row is one requirement mapped to a run, a metric, an operator, and a pass/fail threshold. Your Case Loader reads this file, checks its structural validity, and hands a clean table to the rest of the pipeline.

Strict validation matters here. A harness that silently accepts a malformed input file will produce meaningless results. Named errors, not crashes, let automated tools catch and report problems cleanly.

Which file to open

Open `load_test_cases.m` in MATLAB. This is where you write your Task 1 implementation. Do NOT write your implementation in `lab03_scaffold.m`; the scaffold only calls your function, it does not contain it. The scaffold's TODO 1 block has one line: call `load_test_cases('test_cases.csv')` and assign the result to `cases`. Your work is inside `load_test_cases.m` itself.

The scaffold's TODO comment block will tell you exactly what to call. Your job is to make the function it calls work correctly.

Function signature

```
function cases = load_test_cases(filename)
```

CSV schema: 7 required columns

Column	Type	Example
test_id	string	TC-01
req_id	string	R-RESP-01
description	string	Step 60C rise time
run_id	double	1
metric	string	rise_time_10_90
operator	string	<=
threshold	double	30.0

Valid metrics: `rise_time_10_90`, `ss_error_last20s`, `max_heater_above_85`, `ramp_tracking_lag`.

Valid operators: `<=`, `>=`, `<`, `>`, `==`

Named errors to throw

Condition	Error ID	Message format
File does not exist	MC3113:FileNotFound	'File not found: %s'
Required column absent	MC3113:MissingColumn	'Missing column: %s'
Invalid metric string	MC3113:InvalidMetric	'Invalid metric: %s'
Invalid operator string	MC3113:InvalidOperator	'Invalid operator: %s'

MATLAB patterns for the four checks

Note on TODO numbering: You will notice the guide skips from TODO 1a directly to TODO 1c. There is no TODO 1b for you to write. The `readtable()` block described in the "Critical implementation note" below is the logical 1b step, but it is fully PROVIDED in `load_test_cases.m`. You do not need to find or write TODO 1b; the numbering gap is intentional.

```
% TODO 1a – file existence
if ~isfile(filename)
    error('MC3113:FileNotFound', 'File not found: %s', filename);
end

% TODO 1c – required column presence (do this for each column in REQUIRED_COLS)
for c = REQUIRED_COLS
    if ~ismember(c{1}, cases.Properties.VariableNames)
        error('MC3113:MissingColumn', 'Missing column: %s', c{1});
    end
end

% TODO 1d – metric and operator string validation (do this for each row)
for r = 1:height(cases)
    if ~ismember(cases.metric(r), VALID_METRICS)
        error('MC3113:InvalidMetric', 'Invalid metric: %s', char(cases.metric(r)));
    end
    if ~ismember(cases.operator(r), VALID_OPERATORS)
        error('MC3113:InvalidOperator', 'Invalid operator: %s', char(cases.operator(r)));
    end
end
```

VALID_METRICS and VALID_OPERATORS are already defined at the top of `load_test_cases.m` as string arrays. Use them directly; do not redefine them.

Critical implementation note – the *readtable()* skeleton

load_test_cases.m already contains a PROVIDED block that correctly reads the CSV. Do not delete it or write your own readtable call. The block looks like this:

```
opts = detectImportOptions(filename, 'TextType', 'string');  
opts.VariableNames = strtrim(opts.VariableNames);           % ← the strtrim fix  
opts = setvartype(opts, {'run_id', 'threshold'}, 'double');  
cases = readtable(filename, opts);
```

Why the strtrim fix is there: CSV files saved from Excel or some editors silently add spaces after commas in the header row. Without strtrim, MATLAB reads ' run_id' (with a leading space) as a column name, so ismember('run_id', ...) returns false and triggers a false MissingColumn error on every valid file.

Your TODO blocks go BEFORE this provided block (TODO 1a: file existence check) and AFTER it (TODO 1c: column validation, TODO 1d: metric/operator validation).

Checkpoint 1 (14:20)

```
cases = load_test_cases('test_cases.csv')
```

Expected output: a **3×7 table** with no errors. *cases.Properties.VariableNames* must list all 7 column names exactly.

E.g.: TC-01 R-RESP-01 Step 60C rise time 1 rise_time_10_90 <= 30

6. Task 2: Stimulus Engine (14:20–15:05)

What does it do and why

The controller needs to be driven by a realistic input signal, not just “set the temperature to X”. For a step test, the setpoint jumps instantly and holds. For a ramp test, it rises linearly from ambient to target over the full run duration. For the fault test, it holds at a dangerously high setpoint.

Your Stimulus Engine generates these profiles as MATLAB arrays. The shape of the profile determines which oracle metric is valid; you cannot use rise time on a ramp input (there is no step to rise from), and you cannot use ramp tracking error on a step input.

Which file to open

Open *stimulus_engine.m*. The file already contains the time vector line:

```
t_arr = (0 : params.dt : params.duration)'; % PROVIDED: do not change
```

Your four TODO blocks (2a, 2b, 2c, 2d) go below this line. You implement each stimulus type in an *if/elseif* block. The scaffold's TODO 2 then calls your completed *stimulus_engine* function.

Function signature

```
function [t_arr, sp_arr] = stimulus_engine(stim_type, params)
```


Input params struct fields

Field	Type	Description
t_amb	double	Ambient temperature [°C] (ramp start value)
setpoint	double	Target setpoint [°C]
duration	double	Total run length [s]
dt	double	Time step [s] (always 0.1 s in this lab)

Stimulus behaviour

stim_type	sp_arr shape	MATLAB hint
'step'	Constant at setpoint from t=0	ones(N,1) * params.setpoint
'ramp'	Linear from t_amb at t=0 to setpoint at t=end	linspace(params.t_amb, params.setpoint, N)'
'sustained_fault'	Constant at setpoint (identical shape to step)	ones(N,1) * params.setpoint
unknown	—	error('MC3113:UnknownStimulus', ...)

Understanding the stub structure

Open *stimulus_engine.m*. The file already contains a complete *if/elseif/else* control structure with four branches, one per stimulus type. Do not rewrite this as a switch block, and do not add new branches. Both *if/elseif* and *switch* are equally valid for string comparisons in MATLAB; the stub uses *if/elseif*, and that is correct.

Your only job is to replace the four placeholder lines (*sp_arr = zeros(N,1);* and *sp_arr = [];*) with the correct expressions shown below. Touch nothing else.

Branch already in the file	Your replacement for the placeholder
strcmp(stim_type, 'step')	sp_arr = ones(N, 1) * params.setpoint;
strcmp(stim_type, 'ramp')	sp_arr = linspace(params.t_amb, params.setpoint, N)';
strcmp(stim_type, 'sustained_fault')	sp_arr = ones(N, 1) * params.setpoint;
else (unknown type)	error('MC3113:UnknownStimulus', 'Unknown stim_type: %s', stim_type);

Two things to get right:

- **Ramp transpose:** *linspace(...)* returns a **row** vector. The ' at the end transposes it to a column. Without it, *execute_test* will crash with a dimension mismatch. Do not omit it.
- **else branch:** Delete the *sp_arr = [];* placeholder line entirely after adding the *error(...)* call. MATLAB never reaches it, but leaving it implies a valid output exists for an unknown input type.

Note:

Ramp start value: The ramp must begin at `params.t_amb`, not at 0 °C. Starting from 0 °C introduces a 20 °C offset into the ramp-tracking oracle, falsely inflating the measured error.

sp_arr length: `t_arr` is generated as `(0: params.dt: params.duration)'`. Due to floating-point rounding in `median(diff(...))`, this may differ by ± 1 sample from `height(log_raw)`. The scaffold handles the alignment for you; do not worry about this.

Checkpoint 2 (15:05)

Read before running: The three test calls below use synthetic durations (90 s, 120 s, 180 s) chosen to give round sample counts. The actual run durations decoded from `sensor_stream.bin` are shorter: Run 1 $\approx$ 86 s, Run 2 $\approx$ 115 s, Run 3 $\approx$ 173 s. When `stimulus_engine` is later called by the scaffold on real run data, the sample counts will differ from what you see here. This is expected; do not adjust your code to match the checkpoint counts.

```
p.t_amb=20; p.setpoint=60; p.duration=90; p.dt=0.1;
[t,sp] = stimulus_engine('step', p);
fprintf('step: N=%d sp(1)=%.1f sp(end)=%.1f\n', numel(t), sp(1), sp(end));
% Expected: N=901 sp(1)=60.0 sp(end)=60.0

p.setpoint=80; p.duration=120;
[t,sp] = stimulus_engine('ramp', p);
fprintf('ramp: N=%d sp(1)=%.1f sp(end)=%.1f\n', numel(t), sp(1), sp(end));
% Expected: N=1201 sp(1)=20.0 sp(end)=80.0

p.setpoint=95; p.duration=180;
[t,sp] = stimulus_engine('sustained_fault', p);
fprintf('fault: N=%d sp(1)=%.1f sp(end)=%.1f\n', numel(t), sp(1), sp(end));
% Expected: N=1801 sp(1)=95.0 sp(end)=95.0
```

If the sample counts and endpoint values match, your `stimulus_engine` is correct. **If the counts differ, check your `(0 : params.dt : params.duration)'` formula — the ' transpose is required.**

Fallback Gate (15:05)

At 10:05, run `verify_task1` in the MATLAB command window. Check these four (04) specific lines in its output:

```
CHECK 3 valid frame count : 3741    PASS
CHECK 6 Run 1 row count   : 863 ±5   PASS
CHECK 7 Run 2 row count   : 1151 ±5   PASS
CHECK 8 Run 3 row count   : 1727 ±5   PASS
```

`decode_frames.m` is provided and should not fail. If CHECK 3, 6, 7, or 8 shows FAIL, raise your hand and ask the TA for `decoded_fallback.csv`. Then run:

```
copyfile('decoded_fallback.csv', 'decoded.csv');

decoded = readtable('decoded_fallback.csv', 'TextType', 'string');
decoded.run_id = double(decoded.run_id);
```

Continue immediately from Task 2; do not spend any more time on Task 1. No additional penalty applies beyond whatever Task 1 marks were already lost.

7. Task 3: Test Oracle (15:05–16:00)

What does it do and why

The oracle is the analytical core of the harness. After the stimulus has been generated and the controller has been run, the oracle answers one question: *Does this output meet the requirement?*

It does so by computing a single numeric metric from the controller’s output trace, comparing it against the threshold in the test case, and returning a structured verdict. The margin field tells you how much headroom remains; a positive margin means the controller passed with room to spare; a negative margin means it failed, and by how much.

Which file to open

Open *oracle.m*. The stub already contains:

- The function signature: `function result = oracle(log, tc)`
- A switch block on `tc.metric` with four cases (one per metric)
- The verdict logic (the switch op block); do NOT modify this

Your work is inside each case of the switch `tc.metric` block: compute the measured value for that metric using the formulas below, then assign it to `result.measured`. The stub assigns all other result fields (`test_id`, `metric`, `threshold`, `operator`, `verdict`, `margin`) for you after your measured value is set.

You do not need to write the verdict or margin logic; it is already provided.

Function signature

function result = oracle(log, tc)

Input log table: required columns

Column	Source	Description
t	decoded	Time vector [s]
setpoint_degC	stimulus_engine (overwritten)	Setpoint profile
t_amb_degC	decoded	Ambient temperature [°C]
chamber_temp	execute_test	CUT output: must exist before oracle is called

Output result struct

Field	Type	Description
test_id	string	From tc.test_id

Field	Type	Description
metric	string	Metric name
measured	double	Computed value
threshold	double	From tc.threshold
operator	string	From tc.operator
verdict	string	'PASS' or 'FAIL'
margin	double	threshold – measured (positive = headroom)

A note on margin sign and operator direction

The margin is always computed as threshold – measured, regardless of operator.

What a positive margin means depends on the operator in that test case:

- operator <= positive margin → measured is BELOW threshold → good (headroom)
- operator >= positive margin → measured is BELOW threshold → bad (shortfall)

For all three seed test cases (TC-01 through TC-03), the operator is <=, so a positive margin always means the controller has headroom. If you add a new test case with operator >= (e.g., a minimum efficiency requirement), a positive margin on a FAIL result is not a contradiction; it means the measured value fell short of the required minimum. You do not need to change anything in *oracle.m*; this is purely for your own interpretation of the report's output.

Wire `execute_test` before you implement any metrics

Before the oracle can compute anything, `chamber_temp` must exist in the log table. This happens in the scaffold (*lab03_scaffold.m*), not in *oracle.m*.

COMMON MISTAKE — Do this before implementing any metric formulas.

Open *lab03_scaffold.m* and find the TODO 3 block inside the for-loop:

```
% — TODO 3 – Run the CUT to produce chamber_temp —————
% Call execute_test(log) and assign the result back to log.
% After this line, log.chamber_temp must exist.
% >>> YOUR CODE HERE <<<
```

Add exactly this one line:

```
log = execute_test(log);
```

This runs the sealed *thermal_controller.p* step-by-step across the log's time vector and appends three new columns to the table: *chamber_temp*, *heater_cmd*, and *cooler_cmd*.

If you skip this line: *chamber_temp* will not exist in the log table, oracle will crash or silently return wrong results (all margins +1.0000, TC-03 returns PASS instead of FAIL).

Verify it is wired before moving on to any oracle metric.

The four metrics

rise_time_10_90 — step response only

The time for chamber_temp to rise from 10% to 90% of the full step magnitude.

```
step_mag = setpoint(end) - t_amb
T_10     = t_amb + 0.10 × step_mag
T_90     = t_amb + 0.90 × step_mag
measured = t(first index where chamber_temp ≥ T_90)
          - t(first index where chamber_temp ≥ T_10)
```

If either crossing is never reached, return measured = Inf. Use: find(chamber_temp >= T_val, 1, 'first') — check for empty before using.

ss_error_last20s — steady-state tracking

The mean absolute error between setpoint and chamber temperature over the **last 20 seconds** of the run.

```
window = (t >= t(end) - 20.0)
measured = mean( abs( setpoint(window) - chamber_temp(window) ) )
```

Use a time-anchored window $t \geq t_{\text{end}} - 20$, not the last N rows. The two are equivalent only if dt is exactly constant — it is not always.

max_heater_above_85 — safety fraction

The fraction of timesteps where chamber_temp exceeded 85 °C.

```
measured = sum(chamber_temp > 85) / length(chamber_temp)
```

A value of 1.0 indicates the chamber exceeded 85 °C for the *entire* run. This metric will flag the safety defect in TC-03.

ramp_tracking_lag — ramp following error

The mean absolute error between the ramp setpoint and the chamber temperature, measured over the full run duration (all timesteps, including the initial transient).

```
measured = mean(abs(setpoint - chamber_temp))
```

Do not use rise_time_10_90 for a ramp stimulus. Rise time is mathematically undefined for a ramp input, there is no step to measure a rise from, and find will return an empty index, giving measured = Inf and a false FAIL.

Verdict logic (provided in oracle.m stub — do not change)

The stub already contains the switch op block that evaluates measured against thresholds (<=, >=, etc.) and assigns 'PASS' or 'FAIL'. Your work is the four metric computations above.

TC-03 will return FAIL — this is correct

TC-03 tests max_heater_above_85 <= 0.80 on Run 3 (sustained 95 °C fault). The sealed controller has an **intentional safety defect**: the heater cut-off does not engage correctly above 85 °C. Your oracle should measure approximately **0.83** and return FAIL with a **negative margin** (≈

-0.03). This is the expected and correct result. Do not adjust your threshold. Do not assume your oracle is broken.

Checkpoint 3 (16:00)

Run `lab03_scaffold` in MATLAB. The console must print:

```
TC-01 rise_time_10_90 PASS measured≈9.8 s margin≈+20.2
TC-02 ss_error_last20s PASS measured≈0.05°C margin≈+0.95
TC-03 max_heater_above_85 FAIL measured≈0.83 margin≈-0.03
```

TC-03 FAIL with a **negative margin** confirms your oracle correctly identified the safety defect.

8. Task 4: Report Generator (16:00–16:20)

What does it do and why

A V&V harness is only useful if its results can be consumed by other tools; a CI/CD system, a requirements tracker, or a project manager's dashboard. Hard-coded `fprintf` output cannot be parsed by a machine. Your Report Generator writes two structured output files: a JSON report for machine consumption, and a CSV coverage table for human review.

Which file to open

Open `report_generator.m`. The stub contains:

- TODO 5a: build the report struct and write `lab03_report.json`
- TODO 5b: build the coverage table and write `lab03_coverage.csv`

The scaffold's TODO 5 calls `report_generator(cases, results, 'lab03_report.json', 'lab03_coverage.csv')`. Your work is inside `report_generator.m`.

Function signature

```
function report_generator(cases, results, out_json, out_csv)
```

lab03_report.json structure

The JSON must match this structure exactly (jsondecode must parse it without errors):

```

{
  "generated": "2026-03-24T08:30:00",
  "total_cases": 3,
  "passed": 2,
  "failed": 1,
  "results": [
    {
      "test_id": "TC-01",
      "req_id": "R-RESP-01",
      "description": "Step 60C rise time",
      "metric": "rise_time_10_90",
      "measured": 9.8,
      "threshold": 30.0,
      "operator": "<=",
      "verdict": "PASS",
      "margin": 20.2
    }
  ]
}

```

MATLAB struct skeleton to produce this JSON (TODO 5a)

Build the top-level report struct, then fill a results array in a loop:

```

report.generated = string(datetime('now','Format','yyyy-MM-dd'T'HH:mm:ss'));
report.total_cases = height(cases);
report.passed      = sum(strcmp({results.verdict}, 'PASS'));
report.failed      = sum(strcmp({results.verdict}, 'FAIL'));

for k = 1:height(cases)
    row.test_id      = char(cases.test_id(k));
    row.req_id       = char(cases.req_id(k));      % string VALUE – not field
    name
    row.description   = char(cases.description(k));
    row.metric        = results(k).metric;
    row.measured      = results(k).measured;
    row.threshold     = results(k).threshold;
    row.operator      = results(k).operator;
    row.verdict       = results(k).verdict;
    row.margin        = results(k).margin;
    rows(k) = row;
end
report.results = rows;

json_str = jsonencode(report, 'PrettyPrint', true);
fid = fopen(out_json, 'w');
fprintf(fid, '%s\n', json_str);
fclose(fid);

```

Critical: the hyphen trap

Do NOT use req_id values like R-RESP-01 as MATLAB struct field names:

- BAD: `s('R-RESP-01') = ...` % hyphens crash jsonencode
- GOOD: `row.req_id = 'R-RESP-01';` % string value inside a safe field name

The skeleton above already uses the correct pattern.:

lab03_coverage.csv columns and MATLAB writer (TODO 5b)

Required columns: test_id, req_id, description, metric, verdict, margin

```
cov = table( ...  
    string({results.test_id})',    ... % or from cases  
    cases.req_id,                  ...  
    cases.description,             ...  
    string({results.metric})',     ...  
    string({results.verdict})',    ...  
    [results.margin]',             ...  
    'VariableNames',  
    {'test_id','req_id','description','metric','verdict','margin'});  
writetable(cov, out_csv);
```


Checkpoint 4 (16:20)

```
raw = fileread('lab03_report.json');
data = jsondecode(raw);
fprintf('passed=%d failed=%d\n', data.passed, data.failed);
% Expected: passed=2 failed=1

cov = readtable('lab03_coverage.csv', 'TextType', 'string');
fprintf('%d rows in coverage CSV\n', height(cov));
% Expected: 3 rows (grows to 5 after Task 5)
```

9. Task 5: Add Two New Test Cases (16:20–16:45)

What does it do and why

Three test cases cover three requirements. Real systems have more. Your final task is to extend the test suite yourself, writing two new rows that exercise requirements not yet covered by TC-01 through TC-03. This is the LLO1 moment: you are *operationalising a requirement* from scratch.

Open test_cases.csv and add two new rows. **Constraints:**

- Each new row must use a different **req_id** from the existing three. Valid **req_id** values are any of: **R-RESP-01**, **R-SS-01**, **R-STAB-01**, **R-CONS-01**, **R-SAFE-01** (from cut_interface_spec.pdf), or a new identifier you define (e.g. R-RAMP-01). You do not need to register new **req_id** values anywhere; any unique string is accepted.

Note: your chosen metric must still be one of the four valid metrics in VALID_METRICS (*rise_time_10_90*, *ss_error_last20s*, *max_heater_above_85*, *ramp_tracking_lag*). **R-CONS-01** and **R-STAB-01** do not have dedicated metrics in this lab and cannot be directly operationalized without adding a new metric; do not attempt to use them.

- Each must reference a valid **run_id** (1, 2, or 3)
- **At least one new row must use Run 2 with metric ramp_tracking_lag**
- Thresholds must be physically meaningful — not trivially PASS (threshold=1000) or trivially FAIL (threshold=0). Use your Checkpoint 3 results as anchors:
 - Use your **Checkpoint 3 console output** as your starting anchor but read it carefully; the three seed test cases only cover Run 1 (step) and Run 3 (fault). Run 2 (ramp) is untested until you add it.
 - For a new **Run 2 row using ramp_tracking_lag**: your oracle will measure the mean absolute error between the ramp setpoint and the chamber temperature over the full 115-second run. A ramp test typically produces a small but non-zero tracking lag because the controller has finite bandwidth — the chamber temperature slightly trails the rising setpoint. A physically meaningful threshold is in the range $\leq 3.0\text{ °C}$ to $\leq 8.0\text{ °C}$. A threshold of $\leq 0.01\text{ °C}$ would trivially fail a real controller (no real system tracks a ramp with sub-degree precision); a threshold of $\leq 50.0\text{ °C}$ would trivially pass anything. Reason through what "acceptable ramp tracking" means physically and pick a value in between.
 - For a second new row, you can use any valid combination of *run_id* and metric, including a second Run 1 metric not already covered (e.g. *ss_error_last20s* on a different setpoint run), or a second Run 3 safety check. The same reasoning

applies: anchor your threshold to the measurement's magnitude, not to an arbitrary number.

- **Do not transfer the Run 1 *ss_error_last20s* value (≈ 0.05 °C) directly to a Run 2 threshold.** The Run 2 ramp ends at 80 °C; the controller's steady-state behaviour at the end of a ramp is different from its behaviour after a step. If you set a Run 2 *ss_error* threshold of ≤ 0.05 °C, expect a FAIL; not because your oracle is wrong, but because the threshold is unrealistically tight for a ramp stimulus.

Run 2 reference point: Once your harness is fully working, running it against the seed test cases will also process Run 2 as part of TC-04 or TC-05 (if you add them). As a calibration reference: the controller's *ramp_tracking_lag* on Run 2 is approximately **3–5 °C**. A threshold of ≤ 5.0 °C would PASS; a threshold of ≤ 2.0 °C would FAIL. Both are defensible choices with different engineering interpretations; just state your reasoning in a comment in the CSV row's description field.

These are reference points only; do not copy them verbatim. Your rows must have unique *req_id* values and thresholds you have reasoned through. A threshold of ≤ 0.001 °C would earn zero marks (it trivially passes a near-perfect controller with no engineering margin).

Available run data

run_id	Stimulus	Setpoint	Duration
1	step	60 °C	~86 s
2	ramp	80 °C	~115 s
3	sustained_fault	95 °C	~173 s

After saving the CSV, re-run *lab03_scaffold*. The report must now show **5 test cases total**, and all *req_id* values in the coverage CSV must be unique.

10.Submission (16:45–17:00)

Step 1: Rename scaffold

In MATLAB's Current Folder panel: right-click lab03_scaffold.m → **Rename** → run_harness.m

Step 2: Final verification

`run_harness`

Confirm: 5 test cases printed, TC-03 shows FAIL, JSON and CSV written to the folder.

Step 3: Push to GitHub

Open a terminal (not MATLAB). Navigate to your repo's lab03/ folder.

```
cd path/to/your-repo/lab03
```

Then run:

```
git checkout -b lab03    % creates the lab03 branch (first time only)
git add run_harness.m lab03_report.json lab03_coverage.csv test_cases.csv
git commit -m "Lab 03 submission — [TEAM NAME]"
git push origin lab03
```

If the lab03 branch already exists from a previous attempt, use git checkout lab03 (without -b) instead of git checkout -b lab03. Using -b on an existing branch will produce an error: "fatal: A branch named 'lab03' already exists." Push to the lab03 branch, NOT main. Your submission timestamp is the time of the Git push. The late penalty applies if this is after 18:00.

Four files to submit

File	Must contain
run_harness.m	All TODOs implemented; scaffold renamed
lab03_report.json	failed ≥ 1 ; TC-03 margin is negative
lab03_coverage.csv	5 rows; all req_id values unique
test_cases.csv	5 rows (3 seed + 2 new)

11.Marking Criteria (10 marks)

Component	Marks	What is assessed
Task 1 — Case Loader	2	Schema validation; all four named errors throw correctly
Task 2 — Stimulus Engine	2	Correct profiles; ramp starts at t_{amb} not 0°C
Task 3 — Oracle	3	All metrics correct; TC-03 returns FAIL with strictly negative margin (margin < 0). A margin of exactly 0.000 does not satisfy this criterion.
Task 4 — Report Generator	2	Valid JSON parseable by jsondecode; coverage CSV complete
Task 5 — New Test Cases	1	Two rows; at least one uses ramp_tracking_lag on Run 2

Late penalty: -1 mark per 30 minutes after 18:00. Hard deadline 23:00.

12.FAQ

Q: TC-03 returns FAIL. Did I do something wrong? No. TC-03 is designed to FAIL. The sealed CUT contains an intentional safety defect. A correct oracle always returns FAIL for TC-03. If TC-03 returns PASS, your `max_heater_above_85` implementation is wrong.

Q: I see `fault_flag` mentioned in `cut_interface_spec.pdf`. Should I use it? No. `execute_test.m` does not write `fault_flag` into the log table, so it is not accessible as a column in `oracle.m`. None of the four required metrics use it. You may observe it conceptually, but do not attempt to add it as a metric or column; your code will crash with an "Undefined variable" error if you reference `log.fault_flag`.

Q: My `rise_time_10_90` is Inf, why? `execute_test` has not been called before oracle, so `chamber_temp` is flat (all zeros or matching the setpoint). In `lab03_scaffold.m`, confirm TODO 3 calls `log = execute_test(log)` and assigns the result back to `log` *before* passing it to oracle.

Q: `detectImportOptions` says "Unknown variable name `run_id`" Your CSV has spaces after commas in the header. Add `opts.VariableNames = strtrim(opts.VariableNames);` immediately after `detectImportOptions` and before `readtable`.

Q: `jsonencode` crashes on my struct. A struct field name contains a hyphen (e.g., you wrote `s('R-SAFE-01') = ...`). Store `req_id` as a string *value* inside a field with a legal name: `row(k).req_id = 'R-SAFE-01';`

Q: `thermal_controller` is undefined. `thermal_controller.p` is not on your MATLAB path. Fix it with one of these two methods:

Method A (command line: recommended, works every time): Run this in the MATLAB command window: `addpath(pwd)` This adds the current folder (`lab03/`) to the path for the current session.

Method B (GUI): In the Current Folder panel, right-click the `lab03/` folder (not just the `.p` file) → Add to Path → Selected Folders. Then click "Save" in the Set Path dialog if prompted; otherwise, the path is lost when MATLAB restarts.

If you get the error again after restarting MATLAB, use Method A again. `addpath(pwd)` must be run once each time you start a new MATLAB session for this lab.

Q: All my margins are exactly +1.0000 `chamber_temp` equals setpoint; `execute_test` was never called, so `chamber_temp` is the proxy column (both are equal, error = 0, and the threshold is 1.0). Wire `log = execute_test(log)` in TODO 3 of the scaffold.

Q: `ss_error_last20s` is always 0.0000. Same cause; `execute_test` not called. Fix TODO 3.

Q: Git push is rejected. You are pushing to main, or the `lab03` branch does not exist yet. Run `git checkout -b lab03` first, then push.

13.Troubleshooting

Symptom	Likely cause	Fix
CHECK 2 FAIL in verify_task1	Wrong binary file	Ask the TA for correct sensor_stream.bin
MissingColumnError on load	CSV header has spaces	Add rtrim fix after detectImportOptions
rise_time_10_90 = Inf	execute_test was not called before oracle	Add log = execute_test(log) in TODO 3
TC-03 measured ≈ 0.48 , PASS	Old binary (Run 3 was 60s)	Re-run verify_task1 — CHECK 8 will fail; ask TA
TC-03 measured ≈ 0.83 , FAIL	execute_test called; intentional defect detected	Correct — do nothing
TC-03 measured = exactly 1.000, FAIL	Wrong run slice used (Run 1 step instead of Run 3 fault) — chamber exceeded 85 °C for 100% of all timesteps	Check that your scaffold is using run_id == 3 for TC-03. Re-run verify_task1 to confirm Run 3 has ~1727 rows
jsonencode crashes	Hyphen in struct field name	Store req_id as a string <i>value</i> , not a field name
Git push rejected	Wrong branch or branch missing	git checkout -b lab03 then push
thermal_controller undefined	.p not on MATLAB path	Right-click file → Add to Path
All margins = +1.0000	chamber_temp equals setpoint (proxy)	execute_test not called — fix TODO 3